

Test Case Design Techniques

Five black-box methods for choosing the few tests that matter most — out of the millions you could write.

BVA · Equivalence Partitioning · Decision Tables · State Transition · Use Case

WHY THIS TOPIC EXISTS

You can never test everything. So how do you choose what to test?

A single text box that accepts a number from 1 to 1000 already has thousands of possible inputs — before you even try letters, symbols, or an empty box. Testing all of them is impossible. Testing a random handful is unreliable. Test case design techniques are the disciplined middle path: systematic rules for picking a small set of inputs that finds the most bugs for the least effort.

The goal: maximum defect discovery, minimum number of test cases.

Testing From the Outside In

BLACK-BOX TESTING

Test based only on what the software should DO — its inputs and expected outputs. You don't look at the code inside. Like testing a vending machine by pressing buttons, not opening it up.

WHITE-BOX TESTING

Test based on the internal code and logic — examining the paths, branches, and loops inside. Useful, but a different topic. Today is entirely about the black-box view.

THE FIVE TECHNIQUES WE'LL COVER

1

Boundary Value Analysis

Test the edges

2

Equivalence Partitioning

Group similar inputs

3

Decision Table

Combine conditions

4

State Transition

Follow the states

5

Use Case

Walk the user's journey

TECHNIQUE 1

Boundary Value Analysis

Bugs love edges. Test right where one range ends and another begins.

1

Why Test the Boundaries?

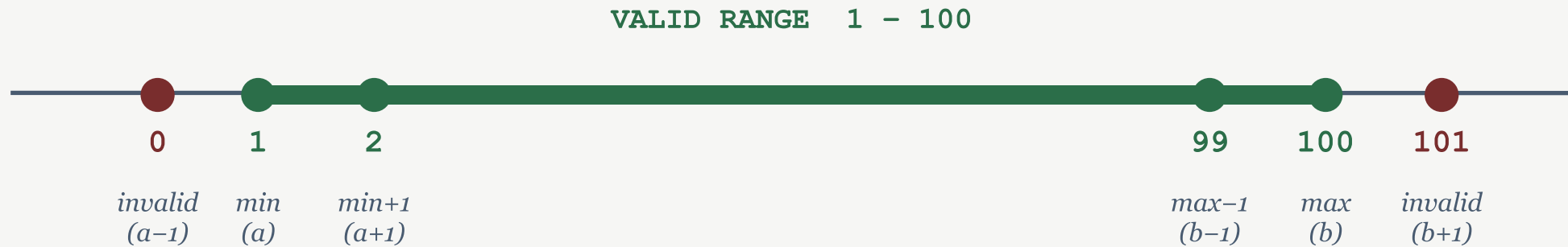
Programmers make mistakes most often at the edges of a range — confusing “less than” with “less than or equal to,” starting a loop at 1 instead of 0, stopping one step too early or too late. These “off-by-one” errors cluster at boundaries. So instead of testing random values from the middle, Boundary Value Analysis deliberately tests the values right at, and just beside, each boundary.

THE RULE

For a valid range from a to b, test six values: $a-1$, a , $a+1$ and $b-1$, b , $b+1$.

In plain words: test just below the edge, exactly on the edge, and just above the edge — at both the lower and upper boundary. Those are the spots where the code is most likely to trip.

Example: An Age Field That Accepts 1–100



SIX TEST CASES, NOT A HUNDRED

Instead of checking all 100 ages, we test just six: 0, 1, 2, 99, 100, 101. Two of them (0 and 101) should be rejected; four should be accepted. If the programmer wrote “> 100” when they meant “≥ 100,” this tiny set catches it immediately.

TECHNIQUE 2

Equivalence Partitioning

If a hundred inputs would all behave the same way, why test more than one of them?

2

Group Inputs That Behave the Same

Many different inputs are treated identically by the software. If entering 45 and entering 67 both simply mean “a valid age,” testing both is wasted effort — they exercise the same logic. Equivalence Partitioning divides all possible inputs into groups (“classes”) where every member is treated the same, then tests just ONE representative from each group.

THE RULE

Split inputs into valid and invalid classes. Test one value from each class — that value “represents” all the others.

BVA and Equivalence Partitioning are best friends: partitioning tells you WHICH groups exist; boundary analysis tells you to test the EDGES of those groups. Used together, they cover a field thoroughly with very few tests.

Example: The Same 1–100 Age Field

All possible inputs fall into just three classes. We need only three tests — one per class.

INVALID (too low)

Anything below 1

Test with: 0

VALID

Anything from 1 to 100

Test with: 50

INVALID (too high)

Anything above 100

Test with: 101

COMBINE THE TWO TECHNIQUES

Equivalence Partitioning alone gives 3 tests (0, 50, 101). Add Boundary Value Analysis and you also check 1, 2, 99, 100. Together: a thorough check of the whole field in about eight test cases — chosen on purpose, not by luck.

TECHNIQUE 3

Decision Table Testing

When the output depends on a combination of conditions, a table guarantees you miss none.

3

When Conditions Combine

Some rules only make sense as combinations. “You may withdraw cash IF the card is valid AND the PIN is correct AND there are sufficient funds.” With three yes/no conditions there are eight possible combinations — and each may need different handling. It's dangerously easy to forget one. A decision table lays out every combination of conditions in columns, and the required action for each, so no case is missed.

THE RULE

List the conditions and actions. Each column is one rule = one unique combination. n conditions $\rightarrow 2^n$ combinations.

Best used when: inputs interact, business rules are complex, or an outcome depends on several things being true at once. Each column becomes one test case.

Example: A Login Screen

Two conditions — correct username? correct password? — give $2^2 = 4$ rules. The table makes all four impossible to forget.

Conditions	Rule 1	Rule 2	Rule 3	Rule 4
Username correct?	Yes	Yes	No	No
Password correct?	Yes	No	Yes	No
Action	Login ✓	Error	Error	Error

Reading it: only Rule 1 — both correct — logs the user in. Every other combination must show an error. Four columns, four test cases, zero forgotten possibilities. (For safety, good systems show the same error for rules 2–4 so attackers can't tell which field was wrong.)

State Transition Testing

Some software behaves differently depending on what happened before. Test the journey, not just the click.



When History Changes Behaviour

The same action can give different results depending on the system's current “state.” Entering a wrong PIN once is fine; entering it three times locks the card. The button press is identical — what changed is the history. State Transition Testing maps out the states a system can be in, the events that move it between them, and checks that every move (and every move that should be blocked) behaves correctly.

STATE

A condition the system is in

e.g. “Logged out”

EVENT

Something that triggers a change

e.g. “Enter password”

TRANSITION

The move from one state to another

Logged out → Logged in

Best used for: login lockouts, ATMs, order status (placed → shipped → delivered), media players, any workflow with steps.

Example: Login Lockout After 3 Wrong Tries



At ANY point before lockout, entering the correct password → transitions straight to “Logged In” and resets the counter to zero.

WHAT THIS FINDS THAT OTHER TECHNIQUES MISS

Testing one login attempt in isolation would never reveal the lockout bug. Only by following the sequence — wrong, wrong, wrong — do we test the transition into “Locked.” We also test what should NOT happen: can a locked card still log in with the right PIN? It must not.

TECHNIQUE 5

Use Case Testing

Zoom out from single fields to the whole journey a real user takes to get something done.

5

Test the Whole User Journey

The other four techniques zoom IN on individual fields and rules. Use Case Testing zooms OUT: it checks that a user can complete a real, end-to-end task from start to finish. A use case is a story of interaction between a user (the “actor”) and the system to achieve a goal — and each realistic path through that story becomes a test.

MAIN FLOW (everything goes right)

Use case: “Buy an item online.”

1. Search for a product
2. Add it to the basket
3. Enter delivery address
4. Pay successfully
5. See order confirmation

ALTERNATE FLOWS (things go wrong)

Each becomes its own test case:

- Payment is declined
- Item goes out of stock mid-checkout
- Address form left incomplete
- User cancels before paying
- Internet drops during payment

Which Technique, When?

Technique	Use it when...	One-line memory hook
Boundary Value Analysis	Inputs have a range with min/max limits	“Test the edges.”
Equivalence Partitioning	Many inputs are treated the same way	“One from each group.”
Decision Table	The result depends on combined conditions	“Every combination in a column.”
State Transition	Behaviour depends on what happened before	“Follow the journey.”
Use Case	You need a full end-to-end user task	“Walk in the user's shoes.”

In real projects, you combine them.

A single checkout page might use partitioning + boundary analysis on the quantity field, a decision table for the discount rules, state transition for the order status, and a use case for the whole purchase. They are teammates, not rivals.

The Importance of Designing Tests

1

Fewer tests, more bugs found

Systematic selection beats random guessing — you catch more defects while writing far fewer test cases. That saves time and money.

2

Coverage you can prove

Techniques give you a defensible answer to “Did you test enough?” — you can show which classes, boundaries, and combinations were covered.

3

Nothing important forgotten

Tables and state maps make gaps visible. You are far less likely to miss the one nasty combination that later becomes a costly production bug.

4

A shared professional language

“Boundary value,” “equivalence class,” “alternate flow” are understood by testers worldwide — and asked about in every QA job interview.